

IFRAME: <https://www.googletagmanager.com/ns.html?id=GTM-T8XT4PS>

[tr?id=243028289693773&ev=PageView&noscript=1]

Skip to main content

(BUTTON) Back to top [] []

_____ Ctrl+K

(BUTTON)

PyTorch - Home

PyTorch - Home

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional
 - + torch.Tensor
 - + Tensor Attributes
 - + Tensor Views
 - + Automatic Mixed Precision package - torch.amp
 - + Automatic differentiation package - torch.autograd
 - + torch.library
 - + torch.accelerator
 - + torch.cpu
 - + torch.cuda
 - + Understanding CUDA Memory Usage
 - + torch.mps
 - + torch.xpu
 - + torch.mtia
 - + torch.mtia.memory
 - + torch.mtia.mtia_graph
 - + Meta device
 - + torch.backends
 - + torch.export
 - + Distributed communication package - torch.distributed
 - + torch.distributed.tensor
 - + Generic Join Context Manager
 - + Torch Distributed Elastic
 - + FullyShardedDataParallel
 - + torch.distributed.fsdp.fully_shard
 - + Tensor Parallelism - torch.distributed.tensor.parallel
 - + Distributed Optimizers
 - + Pipeline Parallelism
 - + PyTorch Symmetric Memory
 - + Distributed Checkpoint - torch.distributed.checkpoint
 - + Probability distributions - torch.distributions
 - + torch.compiler API reference
 - + torch.fft
 - + torch.foreach
 - + torch.func
 - + torch.futures
 - + torch.fx
 - + torch.fx.experimental
 - + torch.hub
 - + torch.linalg
 - + torch.monitor
 - + torch.signal
 - + torch.special
 - + torch.overrides
 - + torch.nativert
 - + torch.package
 - + torch.profiler
 - + torch.nn.init
 - + torch.nn.attention
 - + torch.onnx
 - + torch.optim
 - + Complex Numbers
 - + DDP Communication Hooks
 - + Quantization
 - + Distributed RPC Framework
 - + torch.random

- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCM) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

[]

- * X
- * GitHub
- * PyTorch Forum
- * PyPi

(BUTTON)

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional

- + torch.Tensor
- + Tensor Attributes
- + Tensor Views
- + Automatic Mixed Precision package - torch.amp
- + Automatic differentiation package - torch.autograd
- + torch.library
- + torch.accelerator
- + torch.cpu
- + torch.cuda
- + Understanding CUDA Memory Usage
- + torch.mps
- + torch.xpu
- + torch.mtia
- + torch.mtia.memory
- + torch.mtia.mtia_graph
- + Meta device
- + torch.backends
- + torch.export
- + Distributed communication package - torch.distributed
- + torch.distributed.tensor
- + Generic Join Context Manager
- + Torch Distributed Elastic
- + FullyShardedDataParallel
- + torch.distributed.fsdp.fully_shard
- + Tensor Parallelism - torch.distributed.tensor.parallel
- + Distributed Optimizers
- + Pipeline Parallelism
- + PyTorch Symmetric Memory
- + Distributed Checkpoint - torch.distributed.checkpoint
- + Probability distributions - torch.distributions
- + torch.compiler API reference
- + torch.fft
- + torch.foreach
- + torch.func
- + torch.futures
- + torch.fx
- + torch.fx.experimental
- + torch.hub
- + torch.linalg
- + torch.monitor
- + torch.signal
- + torch.special
- + torch.overrides
- + torch.nativert
- + torch.package
- + torch.profiler
- + torch.nn.init
- + torch.nn.attention
- + torch.onnx
- + torch.optim
- + Complex Numbers
- + DDP Communication Hooks
- + Quantization
- + Distributed RPC Framework
- + torch.random
- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics

- + CPU threading and TorchScript inference
- + CUDA semantics
- + PyTorch Custom Operators Landing Page
- + Distributed Data Parallel
- + Extending PyTorch
- + Extending torch.func with autograd.Function
- + Frequently Asked Questions
- + Getting Started on Intel GPU
- + Gradcheck mechanics
- + HIP (ROCm) semantics
- + Features for large-scale deployments
- + LibTorch Stable ABI
- + LocalTensor Tutorial: Single-Process SPMD Debugging
- + MKLDNN backend
- + Modules
- + MPS backend
- + Multiprocessing best practices
- + Numerical accuracy
- + Out Notes
- + Reproducibility
- + Serialization semantics
- + TensorIterator (Python)
- + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

-
- []
- * X
 - * GitHub
 - * PyTorch Forum
 - * PyPi

Section Navigation

Introduction

- * PyTorch Overview
- * Get Started
- * Learn the Basics

Core Concepts

- * PyTorch Main Components

Torch Compile

- * Torch.compile
 - + Getting Started
 - + Core Concepts
 - o torch.compile Programming Model
 - # Dynamo Core Concepts
 - # Working with Graph Breaks
 - # Non-strict Tracing Programming Model
 - # Dealing with Recompilations
 - # Reducing Compile Time
 - # Reducing Guard Overhead
 - # tparse / TORCH_TRACE
 - # Reporting Issues
 - o Dynamo Overview
 - o PyTorch 2.0 NNModule Support
 - o torch.compile has different autograd semantics
 - + Performance
 - o PyTorch 2.0 Performance Dashboard
 - o TorchInductor GPU Profiling
 - o Profiling to understand torch.compile performance
 - o CUDAGraph Trees
 - + Advanced
 - o Dynamo Deep-Dive
 - o Writing Graph Transformations on ATen IR
 - o Fake tensor
 - o Custom Backends
 - o Dynamic Shapes
 - # Dynamic Shapes Core Concepts
 - # Troubleshooting Dynamic Shapes
 - # Advanced Options to Control Dynamic Behavior

- # Beyond the Basics
 - o Ahead-of-Time Compilation with torch.compile
 - o Complex Number Support in torch.compile
- + Troubleshooting FAQs
 - o tlparsed / TORCH_TRACE
 - o Reporting Issues
 - o torch.compile Troubleshooting
 - o Frequently Asked Questions
- + Reference/API
 - o torch.compiler API reference
 - # torch.compiler.compile
 - # torch.compiler.reset
 - # torch.compiler.nonstrict_trace
 - # torch.compiler.allow_in_graph
 - # torch.compiler.substitute_in_graph
 - # torch.compiler.assume_constant_result
 - # torch.compiler.list_backends
 - # torch.compiler.disable
 - # torch.compiler.set_default_backend
 - # torch.compiler.get_default_backend
 - # torch.compiler.set_stance
 - # torch.compiler.set_enable_guard_collectives
 - # torch.compiler.cudagraph_mark_step_begin
 - # torch.compiler.cudagraph_mark_warmup_incomplete
 - # torch.compiler.is_compiling
 - # torch.compiler.is_dynamo_compiling
 - # torch.compiler.is_exporting
 - # torch.compiler.keep_portable_guards_unsafe
 - # torch.compiler.skip_guard_on_inbuilt_nn_modules_unsafe
 - # torch.compiler.skip_guard_on_all_nn_modules_unsafe
 - # torch.compiler.keep_tensor_guards_unsafe
 - # torch.compiler.skip_guard_on_globals_unsafe
 - # torch.compiler.skip_all_guards_unsafe
 - # torch.compiler.nested_compile_region
 - # torch.compiler.load_cache_artifacts
 - # torch.compiler.load_compiled_function
 - # torch.compiler.save_cache_artifacts
 - # torch.compiler.wrap_numpy
 - o torch.compiler.config
 - o TorchDynamo APIs for fine-grained tracing
 - o TorchInductor and AOTInductor Provenance Tracking
- * Torch.export
 - + torch.export API Reference
 - + torch.export Programming Model
 - + torch.export IR Specification
 - + PT2 Archive Spec
 - + Draft Export
 - + Joint with descriptors
 - + Control Flow Operators
 - o Control Flow - Cond
 - o Control Flow - Switch
 - o Control Flow - While Loop
 - o Control Flow - Scan
 - o Control Flow - Associative Scan
 - o Control Flow - Map
 - + ExportDB
 - o torch.escape-hatch
 - o torch.dynamic-shape
 - o torch.cond
 - o python.closure
 - o torch.dynamic-value
 - o python.data-structure
 - o python.assert
 - o python.control-flow
 - o torch.map
 - o python.builtin
 - o python.object-model
 - o python.context-manager
 - o torch.operator
 - o torch.mutation
 - + AOTInductor: Ahead-Of-Time Compilation for Torch.Export-ed Models
 - o torch._logging
 - # torch._logging.set_logs
 - o AOTInductor Minifier
 - o AOTInductor Debugging Guide
 - + IRs
 - + Dynamic Shapes
 - o Dynamic Shapes Core Concepts
 - o Troubleshooting Dynamic Shapes
 - # Debugging with tlparsed and TORCH_LOGS=dynamic
 - # Troubleshooting GuardOnDataDependentSymNode Errors
 - o Advanced Options to Control Dynamic Behavior

- o Beyond the Basics
 - # The Zero-One Specialization Problem
 - # Backed vs Unbacked Symints
- + Fake tensor
- + Writing Graph Transformations on ATen IR

Developer Notes

- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCm) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ

Accelerator Integration

- * Accelerator Integration
 - + Device Management
 - + Accelerator Hooks
 - + Guard
 - + Autoload Mechanism
 - + Operator Registration
 - + Automatic Mixed Precision
 - + Profiler Integration
 - + Distributed Training Integration
 - + CI Integration

- *
 - * User Guide
 - *
 - * torch.compile Programming Model
 - * Dynamo Core Concepts

Rate this Page

?? ??

Dynamo Core Concepts#

Created On: Jul 28, 2025 | Last Updated On: Jul 27, 2026

Summary:

- * Dynamo, torch.compile's frontend, performs tracing to capture the semantics of a Python function (and its nested function calls) into a linear sequence of operations (the "(FX) graph"), residual bytecode, and "guards" (a list of conditions under which the graph and bytecode are valid).
- * Unsupported Python features lead to graph breaks, where Dynamo compiles a partial graph acquired from tracing, then runs the unsupported code, then resumes tracing.
- * Graph breaks may lead to slowness in torch.compile and prevent backend optimization opportunities. If you're not seeing the performance you expect, then check for graph breaks.

Dynamo Tracing#

torch.compile's frontend (Dynamo) is a custom Python bytecode interpreter designed to allow graph compilation in PyTorch programs while retaining the full flexibility of Python. Given a function to be compiled, Dynamo interprets Python bytecode to extract sequences of PyTorch operations into 1 or more FX graphs that may be further optimized by a backend.

Summary diagram of Dynamo

For example, for the function `f` in the above diagram, Dynamo produces:

- * a single FX graph that takes in the original input plus some additional inputs required by the function.
- * Python bytecode that can be used as a drop-in replacement for `f`. In our example, the bytecode retrieves the additional inputs and passes it to the graph and also contains unoptimizable Python side effects (the list `append`)
- * guards that specify the conditions under which the graph and bytecode are valid. Unless otherwise specified, the graph produced by Dynamo specializes on the shapes of input Tensors.

Graph Breaks#

Dynamo traces your code and attempts to capture your PyTorch code into a single computation graph of PyTorch operators (FX graph). However, this is not always possible. When encountering code that can't be traced, a "graph break" occurs. In the default `torch.compile` settings, a graph break involves compiling the FX graph that has been determined so far, running the unsupported code in regular Python, then resuming tracing after the unsupported code with a new FX graph.

Graph breaks are a feature that allows Dynamo to run over arbitrary Python code and carve out functional subgraphs that can each be individually optimized.

However, it is possible for graph breaks to lead to unexpected slowness in `torch.compile`. If you're not getting the speedups you expect, we recommend checking for graph breaks and removing them.

Graph breaks may occur on things like:

- * Data-dependent if-statements
- * Many Python built-in functions
- * C functions

Below is an example of a graph break due to calling an unsupported operation `torch.save`:

```
@torch.compile
def f(x):
    y = x ** 2 / 2
    torch.save(y, "foo.pt") # torch.save is an unsupported operation
    z = y ** 3 / 6
    return z

x = torch.randn(3)
print(f(x))
```

The semantics of `torch.compile(f)(x)` are roughly this:

```
def compiled_f_semantics(x):
    y = torch.compile(g, fullgraph=True)(x)
    torch.save(y, "foo.pt")
    z = torch.compile(h, fullgraph=True)(x)
    return z

def g(x):
    return x ** 2 / 2

def h(x):
    return y ** 3 / 6
```

Guards#

`torch.compile` makes some assumptions about runtime values as we trace through code. During tracing, we generate "guards", which are runtime checks for these assumptions. Guards are run in future calls to the compiled function to determine if we can reuse previously compiled code. Examples of runtime checks are constant values, types, and object IDs.

Because guards run on every call, they add some per-call overhead. If that overhead is significant for your model, see [Reducing Guard Overhead](#).

Below is an example of generated guards. The `TENSOR_MATCH` guard checks for the input's type, device, dtype, shape, etc.

```
@torch.compile
def fn(x):
    return x + 1

print(fn(torch.ones(3, 3)))
```

Recompilations#

If the guards fail for every instance of previously compiled code, then `torch.compile` must "recompile" the function, requiring the original code to be traced again. In the example below, recompilation is necessary because the guard checking the tensor argument's shape failed. Recompilations add to overall compile time; see

Dealing with Recompilations and Reducing Compile Time for how to reduce that cost.

```
@torch.compile
```

```
def fn(x):  
    return x + 1
```

```
print(fn(torch.ones(3, 3)))  
print(fn(torch.ones(4, 4)))
```

Dynamic Shapes#

`torch.compile` initially assumes tensor shapes are static/constant and guards based on these assumptions. By using "dynamic shapes," we can get `torch.compile` to produce compiled code that can accept tensor inputs with different shapes - we avoid recompiling every time shapes differ. By default, automatic dynamic shapes are enabled in `torch.compile(dynamic=None)` - if compilation fails due to shape mismatch, recompilation is attempted

with dynamic shapes. Dynamic shapes can also be fully enabled (`dynamic=True`) or disabled (`dynamic=False`).

Below, we enable dynamic shapes and note that we no longer need to recompile.

```
@torch.compile(dynamic=True)
```

```
def fn(x):  
    return x + 1
```

```
print(fn(torch.ones(3, 3)))  
print(fn(torch.ones(4, 4)))
```

For more information on dynamic shapes, see The dynamic shapes manual.

Rate this Page

?? ??

(BUTTON) Send Feedback

previous

torch.compile Programming Model

next

Working with Graph Breaks

Built with the PyData Sphinx Theme 0.15.4.

previous

torch.compile Programming Model

next

Working with Graph Breaks

On this page

- * Dynamo Tracing
- * Graph Breaks
- * Guards
- * Recompilations
- * Dynamic Shapes

Edit on GitHub

Show Source

PyTorch Libraries

- * ExecuTorch
- * Helion
- * torchao
- * kineto
- * torchtitan
- * TorchRL
- * torchvision
- * torchaudio
- * tensordict
- * PyTorch on XLA Devices

Access comprehensive developer documentation for PyTorch
[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers
[View Tutorials](#)

Resources

Find development resources and get your questions answered
[View Resources](#)

Stay in touch for updates, event info, and the latest news

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).

*
*
*
*
*
*

(C) PyTorch. Copyright (C) The Linux Foundation(R). All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Trademark Usage](#). [Privacy Policy](#).

To analyze traffic and optimize your experience, we serve cookies on this site. By clicking or navigating, you agree to allow our usage of cookies. As the current maintainers of this site, Facebook's Cookies Policy applies. Learn more, including about available controls: [Cookies Policy](#).
(BUTTON)

(C) Copyright PyTorch Contributors.

Created using Sphinx 7.2.6.

Built with the PyData Sphinx Theme 0.15.4.

IFRAME: <https://www.googletagmanager.com/ns.html?id=GTM-T8XT4PS>

[tr?id=243028289693773&ev=PageView&noscript=1]

Skip to main content

(BUTTON) Back to top [] []

_____ Ctrl+K

(BUTTON)

PyTorch - Home

PyTorch - Home

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional
 - + torch.Tensor
 - + Tensor Attributes
 - + Tensor Views
 - + Automatic Mixed Precision package - torch.amp
 - + Automatic differentiation package - torch.autograd
 - + torch.library
 - + torch.accelerator
 - + torch.cpu
 - + torch.cuda
 - + Understanding CUDA Memory Usage
 - + torch.mps
 - + torch.xpu
 - + torch.mtia
 - + torch.mtia.memory
 - + torch.mtia.mtia_graph
 - + Meta device
 - + torch.backends
 - + torch.export
 - + Distributed communication package - torch.distributed
 - + torch.distributed.tensor
 - + Generic Join Context Manager
 - + Torch Distributed Elastic
 - + FullyShardedDataParallel
 - + torch.distributed.fsdp.fully_shard
 - + Tensor Parallelism - torch.distributed.tensor.parallel
 - + Distributed Optimizers
 - + Pipeline Parallelism
 - + PyTorch Symmetric Memory
 - + Distributed Checkpoint - torch.distributed.checkpoint
 - + Probability distributions - torch.distributions
 - + torch.compiler API reference
 - + torch.fft
 - + torch.foreach
 - + torch.func
 - + torch.futures
 - + torch.fx
 - + torch.fx.experimental
 - + torch.hub
 - + torch.linalg
 - + torch.monitor
 - + torch.signal
 - + torch.special
 - + torch.overrides
 - + torch.nativert
 - + torch.package
 - + torch.profiler
 - + torch.nn.init
 - + torch.nn.attention
 - + torch.onnx
 - + torch.optim
 - + Complex Numbers
 - + DDP Communication Hooks
 - + Quantization
 - + Distributed RPC Framework
 - + torch.random

- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCm) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

[]

- * X
- * GitHub
- * PyTorch Forum
- * PyPi

(BUTTON)

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional

- + torch.Tensor
- + Tensor Attributes
- + Tensor Views
- + Automatic Mixed Precision package - torch.amp
- + Automatic differentiation package - torch.autograd
- + torch.library
- + torch.accelerator
- + torch.cpu
- + torch.cuda
- + Understanding CUDA Memory Usage
- + torch.mps
- + torch.xpu
- + torch.mtia
- + torch.mtia.memory
- + torch.mtia.mtia_graph
- + Meta device
- + torch.backends
- + torch.export
- + Distributed communication package - torch.distributed
- + torch.distributed.tensor
- + Generic Join Context Manager
- + Torch Distributed Elastic
- + FullyShardedDataParallel
- + torch.distributed.fsdp.fully_shard
- + Tensor Parallelism - torch.distributed.tensor.parallel
- + Distributed Optimizers
- + Pipeline Parallelism
- + PyTorch Symmetric Memory
- + Distributed Checkpoint - torch.distributed.checkpoint
- + Probability distributions - torch.distributions
- + torch.compiler API reference
- + torch.fft
- + torch.foreach
- + torch.func
- + torch.futures
- + torch.fx
- + torch.fx.experimental
- + torch.hub
- + torch.linalg
- + torch.monitor
- + torch.signal
- + torch.special
- + torch.overrides
- + torch.nativert
- + torch.package
- + torch.profiler
- + torch.nn.init
- + torch.nn.attention
- + torch.onnx
- + torch.optim
- + Complex Numbers
- + DDP Communication Hooks
- + Quantization
- + Distributed RPC Framework
- + torch.random
- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics

- + CPU threading and TorchScript inference
- + CUDA semantics
- + PyTorch Custom Operators Landing Page
- + Distributed Data Parallel
- + Extending PyTorch
- + Extending torch.func with autograd.Function
- + Frequently Asked Questions
- + Getting Started on Intel GPU
- + Gradcheck mechanics
- + HIP (ROCm) semantics
- + Features for large-scale deployments
- + LibTorch Stable ABI
- + LocalTensor Tutorial: Single-Process SPMD Debugging
- + MKLDNN backend
- + Modules
- + MPS backend
- + Multiprocessing best practices
- + Numerical accuracy
- + Out Notes
- + Reproducibility
- + Serialization semantics
- + TensorIterator (Python)
- + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

-
- []
- * X
 - * GitHub
 - * PyTorch Forum
 - * PyPi

Section Navigation

Introduction

- * PyTorch Overview
- * Get Started
- * Learn the Basics

Core Concepts

- * PyTorch Main Components

Torch Compile

- * Torch.compile
 - + Getting Started
 - + Core Concepts
 - o torch.compile Programming Model
 - # Dynamo Core Concepts
 - # Working with Graph Breaks
 - # Non-strict Tracing Programming Model
 - # Dealing with Recompilations
 - # Reducing Compile Time
 - # Reducing Guard Overhead
 - # tparse / TORCH_TRACE
 - # Reporting Issues
 - o Dynamo Overview
 - o PyTorch 2.0 NNModule Support
 - o torch.compile has different autograd semantics
 - + Performance
 - o PyTorch 2.0 Performance Dashboard
 - o TorchInductor GPU Profiling
 - o Profiling to understand torch.compile performance
 - o CUDAGraph Trees
 - + Advanced
 - o Dynamo Deep-Dive
 - o Writing Graph Transformations on ATen IR
 - o Fake tensor
 - o Custom Backends
 - o Dynamic Shapes
 - # Dynamic Shapes Core Concepts
 - # Troubleshooting Dynamic Shapes
 - # Advanced Options to Control Dynamic Behavior

- # Beyond the Basics
 - o Ahead-of-Time Compilation with torch.compile
 - o Complex Number Support in torch.compile
- + Troubleshooting FAQs
 - o tlparsed / TORCH_TRACE
 - o Reporting Issues
 - o torch.compile Troubleshooting
 - o Frequently Asked Questions
- + Reference/API
 - o torch.compiler API reference
 - # torch.compiler.compile
 - # torch.compiler.reset
 - # torch.compiler.nonstrict_trace
 - # torch.compiler.allow_in_graph
 - # torch.compiler.substitute_in_graph
 - # torch.compiler.assume_constant_result
 - # torch.compiler.list_backends
 - # torch.compiler.disable
 - # torch.compiler.set_default_backend
 - # torch.compiler.get_default_backend
 - # torch.compiler.set_stance
 - # torch.compiler.set_enable_guard_collectives
 - # torch.compiler.cudagraph_mark_step_begin
 - # torch.compiler.cudagraph_mark_warmup_incomplete
 - # torch.compiler.is_compiling
 - # torch.compiler.is_dynamo_compiling
 - # torch.compiler.is_exporting
 - # torch.compiler.keep_portable_guards_unsafe
 - # torch.compiler.skip_guard_on_inbuilt_nn_modules_unsafe
 - # torch.compiler.skip_guard_on_all_nn_modules_unsafe
 - # torch.compiler.keep_tensor_guards_unsafe
 - # torch.compiler.skip_guard_on_globals_unsafe
 - # torch.compiler.skip_all_guards_unsafe
 - # torch.compiler.nested_compile_region
 - # torch.compiler.load_cache_artifacts
 - # torch.compiler.load_compiled_function
 - # torch.compiler.save_cache_artifacts
 - # torch.compiler.wrap_numpy
 - o torch.compiler.config
 - o TorchDynamo APIs for fine-grained tracing
 - o TorchInductor and AOTInductor Provenance Tracking
- * Torch.export
 - + torch.export API Reference
 - + torch.export Programming Model
 - + torch.export IR Specification
 - + PT2 Archive Spec
 - + Draft Export
 - + Joint with descriptors
 - + Control Flow Operators
 - o Control Flow - Cond
 - o Control Flow - Switch
 - o Control Flow - While Loop
 - o Control Flow - Scan
 - o Control Flow - Associative Scan
 - o Control Flow - Map
 - + ExportDB
 - o torch.escape-hatch
 - o torch.dynamic-shape
 - o torch.cond
 - o python.closure
 - o torch.dynamic-value
 - o python.data-structure
 - o python.assert
 - o python.control-flow
 - o torch.map
 - o python.builtin
 - o python.object-model
 - o python.context-manager
 - o torch.operator
 - o torch.mutation
 - + AOTInductor: Ahead-Of-Time Compilation for Torch.Export-ed Models
 - o torch._logging
 - # torch._logging.set_logs
 - o AOTInductor Minifier
 - o AOTInductor Debugging Guide
 - + IRs
 - + Dynamic Shapes
 - o Dynamic Shapes Core Concepts
 - o Troubleshooting Dynamic Shapes
 - # Debugging with tlparsed and TORCH_LOGS=dynamic
 - # Troubleshooting GuardOnDataDependentSymNode Errors
 - o Advanced Options to Control Dynamic Behavior

- o Beyond the Basics
 - # The Zero-One Specialization Problem
 - # Backed vs Unbacked Symints
- + Fake tensor
- + Writing Graph Transformations on ATen IR

Developer Notes

- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCm) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ

Accelerator Integration

- * Accelerator Integration
 - + Device Management
 - + Accelerator Hooks
 - + Guard
 - + Autoload Mechanism
 - + Operator Registration
 - + Automatic Mixed Precision
 - + Profiler Integration
 - + Distributed Training Integration
 - + CI Integration

- *
 - * User Guide
 - * torch.export
 - * torch.export...

Rate this Page
 ? ? ? ? ?

torch.export Programming Model#

Created On: Dec 18, 2024 | Last Updated On: Jan 08, 2026

This document aims to explain the behaviors and capabilities of `torch.export.export()`. It is intended to help build your intuition for how `torch.export.export()` handles code.

Basics of Tracing#

`torch.export.export()` captures a graph representing your model by tracing its execution on "example" inputs and recording the PyTorch operations and conditions observed along the traced path. This graph can then be run on different inputs as long as they satisfy the same conditions.

The basic output of `torch.export.export()` is a single graph of PyTorch operations, with associated metadata.

The exact format of this output is covered in the export IR spec.

Strict vs. Non-Strict Tracing#

`torch.export.export()` provides two modes of tracing.

In non-strict mode, we trace through the program using the normal Python interpreter. Your code executes exactly as it would in eager mode; the only difference is that all Tensors are replaced by fake Tensors, which

have shapes and other forms of metadata but no data, wrapped in Proxy objects that record all operations on them into a graph. We also capture conditions on Tensor shapes that guard the correctness of the generated code.

In strict mode, we first trace through the program using TorchDynamo, a Python bytecode analysis engine. TorchDynamo does not actually execute your Python code. Instead, it symbolically analyzes it and builds a graph based on the results. On the one hand, this analysis allows `torch.export.export()` to provide additional guarantees on Python-level safety (beyond capturing conditions on Tensor shapes, as in non-strict mode). On the other hand, not all Python features are supported by this analysis.

Although currently the default mode of tracing is strict, we strongly recommend using non-strict, which will soon become the default. For most models, conditions on Tensor shapes are enough for soundness, and the additional guarantees on Python-level safety have no impact; at the same time, the possibility of hitting unsupported Python features in TorchDynamo presents an unnecessary risk.

In the rest of this document we assume we are tracing in non-strict mode; in particular, we assume that all Python features are supported.

Values: Static vs. Dynamic#

A key concept in understanding the behavior of `torch.export.export()` is the difference between static and dynamic values.

Static Values#

A static value is a value that is fixed at export time and cannot change between executions of the exported program. When the value is encountered during tracing, we treat it as a constant and hard-code it into the graph.

When an operation is performed (e.g. `x + y`) and all inputs are static, the output of the operation is directly hard-coded into the graph and the operation does not show up (i.e. it gets "constant-folded").

When a value has been hard-coded into the graph, we say that the graph has been specialized to that value. For example:

```
import torch

class MyMod(torch.nn.Module):
    def forward(self, x, y):
        z = y + 7
        return x + z

m = torch.export.export(MyMod(), (torch.randn(1), 3))
print(m.graph_module.code)

"""
def forward(self, arg0_1, arg1_1):
    add = torch.ops.aten.add.Tensor(arg0_1, 10); arg0_1 = None
    return (add,)
"""
```

Here, we provide 3 as the traced value for `y`; it is treated as a static value and added to 7, burning in the static value 10 in the graph.

Dynamic Values#

A dynamic value is one that can change from run to run. It behaves just like a "normal" function argument: you can pass different inputs and expect your function to do the right thing.

Which values are static vs. dynamic?#

Whether a value is static or dynamic depends on its type:

- * For Tensor:
 - + Tensor data is treated as dynamic.
 - + Tensor shapes can be treated by the system as static or dynamic.
 - o By default, shapes of all input Tensors are considered static. The user can override this behavior for any input Tensor by specifying a dynamic shape for it.
 - o Tensors that are part of module state, i.e., parameters and buffers, always have static shapes.
 - + Other forms of Tensor metadata (e.g. device, dtype) are static.
- * Python primitives (`int`, `float`, `bool`, `str`, `None`) are static.

```

    + There are dynamic variants for some primitive types (SymInt, SymFloat, SymBool). Typically users
do
    not have to deal with them.
    + Users can specify integer inputs as dynamic by specifying a dynamic shape for it.
* For Python standard containers (list, tuple, dict, namedtuple):
    + The structure (i.e., length for list and tuple values, and key sequence for dict and namedtuple
    values) is static.
    + The contained elements have these rules applied to them recursively (basically the PyTree scheme)
with
    leaves that are either Tensor or primitive types.
* Other classes (including data classes) can be registered with PyTree (see below), and follow the same
rules
    as the standard containers.

```

Input types#

```

Inputs will be treated as either static or dynamic, based on their type (as explained above).
* A static input will get hard-coded into the graph, and passing a different value at run time will resu
lt in
    an error. Recall that these are mostly values of primitive types.
* A dynamic input behaves like a "normal" function input. Recall that these are mostly values of Tensor
types.

```

By default, the types of inputs you can use for your program are:

- * Tensor
- * Python primitives (int, float, bool, str, None)
- * Python standard containers (list, tuple, dict, namedtuple)

Custom Input Types (PyTree)#

In addition, you can also define your own (custom) class and use it as an input type, but you will need to register such a class as a PyTree.

Here's an example of using an utility to register a dataclass that is used as an input type.

```

@dataclass
class Input:
    f: torch.Tensor
    p: torch.Tensor

import torch.utils._pytree as pytree
pytree.register_dataclass(Input)

class M(torch.nn.Module):
    def forward(self, x: Input):
        return x.f + 1

torch.export.export(M(), (Input(f=torch.ones(10, 4), p=torch.zeros(10, 4)),))

```

Optional input types#

For optional inputs to the program that are not passed in, `torch.export.export()` will specialize to their default values. As a result, the exported program will require users to explicitly pass in all arguments, and will lose the defaulting behavior. For example:

```

class M(torch.nn.Module):
    def forward(self, x, y=None):
        if y is not None:
            return y * x
        return x + x

# Optional input is passed in
ep = torch.export.export(M(), (torch.randn(3, 3), torch.randn(3, 3)))
print(ep)
"""
ExportedProgram:
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 3]", y: "f32[3, 3]"):
        # File: /data/users/angelayi/pytorch/moo.py:15 in forward, code: return y * x
        mul: "f32[3, 3]" = torch.ops.aten.mul.Tensor(y, x); y = x = None
        return (mul,)
"""

# Optional input is not passed in
ep = torch.export.export(M(), (torch.randn(3, 3),))
print(ep)
"""
ExportedProgram:
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 3]", y):
        # File: /data/users/angelayi/pytorch/moo.py:16 in forward, code: return x + x

```

```

add: "f32[3, 3]" = torch.ops.aten.add.Tensor(x, x);  x = None
return (add,)
"""

```

Control Flow: Static vs. Dynamic#

Control flow is supported by `torch.export.export()`. The behavior of control flow depends on whether the value you are branching on is static or dynamic.

Static Control Flow#

Python control flow over static values is supported transparently. (Recall that static values include static shapes, so control flow over static shapes is also covered by this case.)

As mentioned above, we "burn in" static values, so the exported graph will never see any control flow over static values.

In the case of an if statement, we will continue tracing the branch taken at export time. In the case of a for or while statement, we will continue tracing by unrolling the loop.

Dynamic Control Flow: Shape-Dependent vs. Data-Dependent#

When the value involved in a control flow is dynamic, it could depend on dynamic shapes or dynamic data. Given that the compiler traces with information on shapes rather than data, the implications on the programming model are different in these cases.

Dynamic Shape-Dependent Control Flow#

When the value involved in a control flow is a dynamic shape, in most cases we will also know the concrete value of the dynamic shape during tracing: see the following section for more details on how the compiler tracks this information.

In these cases we say that the control flow is shape-dependent. We use the concrete value of the dynamic shape to evaluate the condition to either True or False and continue tracing (as discussed above), additionally emitting a guard corresponding to the condition just evaluated.

Otherwise the control flow is considered data-dependent. We cannot evaluate the condition to either True or False, so cannot continue tracing and have to raise an error at export time. See next section.

Dynamic Data-Dependent Control Flow#

Data-dependent control flow over dynamic values is supported, but you must use one of PyTorch's explicit operators to continue tracing. Using Python control flow statements over dynamic values is not permitted, because the compiler cannot evaluate the conditions necessary to continue tracing and thus an error must be raised at export time.

We provide operators to express general conditionals and loops over dynamic values, e.g., `torch.cond`, `torch.map`. Note that you only need to use these if you truly want data-dependent control flow.

Here's an example of an if statement on a data-dependent condition, `x.sum() > 0`, where `x` is an input Tensor, rewritten using `torch.cond`. Instead of having to decide which branch to trace, now both branches are traced.

```

class M_old(torch.nn.Module):
    def forward(self, x):
        if x.sum() > 0:
            return x.sin()
        else:
            return x.cos()

class M_new(torch.nn.Module):
    def forward(self, x):
        return torch.cond(
            pred=x.sum() > 0,
            true_fn=lambda x: x.sin(),
            false_fn=lambda x: x.cos(),
            operands=(x,)
        )

```

A special case of data-dependent control flow is where it involves a data-dependent dynamic shape: typically, the shape of some intermediate Tensor that depends on input data rather than on input shapes (thus not shape-dependent). Instead of using a control flow operator, in this case you can provide an assertion that

decides whether the condition is True or False. Given such an assertion, we can continue tracing, emitting a guard as above.

We provide operators to express assertions on dynamic shapes, e.g., `torch._check`. Note that you only need to use this when there is control flow on data-dependent dynamic shapes.

Here's an example of an if statement on a condition involving a data-dependent dynamic shape, `nz.shape[0] > 0`, where `nz` is the result of calling `torch.nonzero()`, an operator whose output shape depends on input data. Instead of rewriting it, you can add an assertion using `torch._check` to effectively decide which branch to trace.

```
class M_old(torch.nn.Module):
    def forward(self, x):
        nz = x.nonzero()
        if nz.shape[0] > 0:
            return x.sin()
        else:
            return x.cos()

class M_new(torch.nn.Module):
    def forward(self, x):
        nz = x.nonzero()
        torch._check(nz.shape[0] > 0)
        if nz.shape[0] > 0:
            return x.sin()
        else:
            return x.cos()
```

Basics of Symbolic Shapes#

During tracing, dynamic Tensor shapes and conditions over them are encoded as "symbolic expressions." (In contrast, static Tensor shapes and conditions over them are simply int and bool values.)

A symbol is like a variable; it describes a dynamic Tensor shape.

As tracing proceeds, shapes of intermediate Tensors may be described by more general expressions, typically involving integer arithmetic operators. This is because for most PyTorch operators, shapes of output Tensors can be described as functions of shapes of input Tensors. For example, the shape of the output of `torch.cat()` is the sum of the shapes of its inputs.

Moreover, as we encounter control flow in the program, we create boolean expressions, typically involving relational operators, describing conditions along the traced path. These expressions are evaluated to decide which path to trace through the program, and recorded in a shape environment to guard the correctness of the traced path and to evaluate subsequently created expressions.

We briefly introduce these subsystems next.

Fake Implementations of PyTorch Operators#

Recall that during tracing, we are executing the program with fake Tensors, which have no data. In general we cannot call the actual implementations of PyTorch operators with fake Tensors. Thus each operator needs to have an additional fake (a.k.a. "meta") implementation, which inputs and outputs fake Tensors, that matches the behavior of the actual implementation in terms of shapes and other forms of metadata carried by fake Tensors.

For example, note how the fake implementation of `torch.index_select()` computes the shape of the output using the shape of the input (while ignoring input data and returning empty output data).

```
def meta_index_select(self, dim, index):
    result_size = list(self.size())
    if self.dim() > 0:
        result_size[dim] = index.numel()
    return self.new_empty(result_size)
```

Shape Propagation: Backed vs. Unbacked Dynamic Shapes#

Shapes are propagated using fake implementations of PyTorch operators.

A key concept to understand the propagation of dynamic shapes in particular is the difference between backed and unbacked dynamic shapes: we know the concrete values of the former but not the latter.

Propagation of shapes, including tracking backed and unbacked dynamic shapes, proceeds as follows:

- * The shapes of Tensors representing inputs can be static or dynamic. When dynamic, they are described by symbols; moreover, such symbols are backed since we also know their concrete values given the "real" example inputs provided by the user at export time.
- * The output shape of an operator is computed by its fake implementation, and is either static or dynamic.

When dynamic, in general it is described by a symbolic expression. Moreover:

- + If the output shape depends only on input shapes, it is either static or backed dynamic whenever input shapes are all static or backed dynamic.
- + On the other hand, if the output shape depends on input data, it is necessarily dynamic, and moreover, because we cannot know its concrete value it is unbacked.

Control Flow: Guards and Assertions#

When a condition on shapes is encountered, it either involves only static shapes, in which case it is a boolean, or it involves dynamic shapes, in which case it is a symbolic boolean expression. For the latter:

- * When the condition involves only backed dynamic shapes, we can use the concrete values of those dynamic shapes to evaluate the condition to True or False. We can then add a guard to the shape environment that states that the corresponding symbolic boolean expression is True or False, and continue tracing.

* Otherwise the condition involves unbacked dynamic shapes. In general we cannot evaluate such a condition without additional information; thus we cannot continue tracing, and we must raise an error at export time.

The user is expected to use an explicit PyTorch operator for tracing to continue. This information is added as a guard in the shape environment, and can also possibly help evaluate other subsequently encountered conditions to True or False.

Once the model is exported, any guards on backed dynamic shapes can be understood as conditions on input dynamic shapes. These are verified against a dynamic shape specification that must have been provided to export, describing conditions on dynamic shapes that not only example inputs but also all future inputs are

expected to satisfy for the generated code to be correct. More precisely, the dynamic shape specification must logically imply the generated guards, otherwise an error is raised at export time (along with suggested fixes to the dynamic shape specification). On the other hand, when there are no generated guards on backed dynamic shapes (in particular, when all shapes are static) no dynamic shape specification needs to be provided to export. In general, the dynamic shape specification is converted to runtime assertions on the inputs of the generated code.

Finally, any guards on unbacked dynamic shapes are converted to "inline" runtime assertions. These are added in the generated code at the locations where those unbacked dynamic shapes were created: typically, right after data-dependent operator calls.

Allowed PyTorch operators#

All PyTorch operators are permitted.

Custom operators#

In addition, you can define and use custom operators. Defining a custom operator includes defining a fake implementation for it, just like any other PyTorch operator (see previous section).

Here's an example of a custom sin operator that wraps NumPy, and its registered (trivial) fake implementation.

```
@torch.library.custom_op("mylib::sin", mutates_args=())
def sin(x: Tensor) -> Tensor:
    x_np = x.numpy()
    y_np = np.sin(x_np)
    return torch.from_numpy(y_np)

@torch.library.register_fake("mylib::sin")
def _(x: Tensor) -> Tensor:
    return torch.empty_like(x)
```

Sometimes your custom operator's fake implementation will involve data-dependent shapes. Here's how a fake implementation for a custom nonzero might look like.

...

```
@torch.library.register_fake("mylib::custom_nonzero")
def _(x):
    nnz = torch.library.get_ctx().new_dynamic_size()
    shape = [nnz, x.dim()]
    return x.new_empty(shape, dtype=torch.int64)
```

Module State: Reads vs. Updates#

Module states include parameters, buffers, and regular attributes.

- * A regular attribute can be of any type.
- * On the other hand, parameters and buffers are always Tensors.

Module states can be dynamic or static, based on their types as outlined above. For example, `self.training` is a `bool`, which means it is static; on the other hand, any parameter or buffer is dynamic.

The shapes of any Tensors contained in module states cannot be dynamic, i.e., those shapes are fixed at export time, and cannot change between executions of the exported program.

Access rules#

All module states must be initialized. Accessing a module state that is not already initialized causes an error to be raised at export time.

Reading module states is always permitted.

Updating module states is possible, but must follow the rules below:

- * A static regular attribute (e.g., of primitive type) can be updated. Reads and updates can be freely interleaved, and as expected, any reads will always see the values of the latest updates. Because these attributes are static, we will also burn the values in, so the generated code will not have any instructions to actually "get" or "set" such attributes.
 - * A dynamic regular attribute (e.g., of Tensor type) cannot be updated. To do so, it must be registered as a buffer during module initialization.
 - * A buffer can be updated, where the updating can be in-place (e.g., `self.buffer[:] = ...`) or not (e.g., `self.buffer = ...`).
 - * A parameter cannot be updated. Typically parameters are updated only during training, not during inference.
- We recommend exporting with `torch.no_grad()` to avoid parameter updates at export time.

Effects of functionalization#

Any dynamic module state that is read and/or updated is "lifted" (respectively) as an input and/or output of the generated code.

The exported program stores, along with the generated code, the initial values of parameters and buffers and the constant values of other Tensor attributes.

Rate this Page
? ? ? ? ?
(BUTTON) Send Feedback

[previous](#)

[torch.export API Reference](#)

[next](#)

[torch.export IR Specification](#)

Built with the PyData Sphinx Theme 0.15.4.

[previous](#)

[torch.export API Reference](#)

[next](#)

[torch.export IR Specification](#)

On this page

- * Basics of Tracing
 - + Strict vs. Non-Strict Tracing
- * Values: Static vs. Dynamic

- + Static Values
- + Dynamic Values
- + Which values are static vs. dynamic?
- * Input types
 - + Custom Input Types (PyTree)
 - + Optional input types
- * Control Flow: Static vs. Dynamic
 - + Static Control Flow
 - + Dynamic Control Flow: Shape-Dependent vs. Data-Dependent
 - o Dynamic Shape-Dependent Control Flow
 - o Dynamic Data-Dependent Control Flow
- * Basics of Symbolic Shapes
 - + Fake Implementations of PyTorch Operators
 - o Shape Propagation: Backed vs. Unbacked Dynamic Shapes
 - + Control Flow: Guards and Assertions
- * Allowed PyTorch operators
 - + Custom operators
- * Module State: Reads vs. Updates
 - + Access rules
 - + Effects of functionalization

[Edit on GitHub](#)

[Show Source](#)

[PyTorch Libraries](#)

- * ExecuTorch
- * Helion
- * torchao
- * kineto
- * torchtitan
- * TorchRL
- * torchvision
- * torchaudio
- * tensordict
- * PyTorch on XLA Devices

Docs

[Access comprehensive developer documentation for PyTorch](#)

[View Docs](#)

Tutorials

[Get in-depth tutorials for beginners and advanced developers](#)

[View Tutorials](#)

Resources

[Find development resources and get your questions answered](#)

[View Resources](#)

[Stay in touch for updates, event info, and the latest news](#)

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).

*
*
*
*
*
*

(C) PyTorch. Copyright (C) The Linux Foundation(R). All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Trademark Usage](#). [Privacy Policy](#).

To analyze traffic and optimize your experience, we serve cookies on this site. By clicking or navigating, you agree to allow our usage of cookies. As the current maintainers of this site, Facebook's Cookies Policy applies. [Learn more](#), including about available controls: [Cookies Policy](#).
(BUTTON)

(C) Copyright PyTorch Contributors.

Created using Sphinx 7.2.6.

Built with the PyData Sphinx Theme 0.15.4.

IFRAME: <https://www.googletagmanager.com/ns.html?id=GTM-T8XT4PS>

[tr?id=243028289693773&ev=PageView&noscript=1]

Skip to main content

(BUTTON) Back to top [] []

_____ Ctrl+K

(BUTTON)

PyTorch - Home

PyTorch - Home

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional
 - + torch.Tensor
 - + Tensor Attributes
 - + Tensor Views
 - + Automatic Mixed Precision package - torch.amp
 - + Automatic differentiation package - torch.autograd
 - + torch.library
 - + torch.accelerator
 - + torch.cpu
 - + torch.cuda
 - + Understanding CUDA Memory Usage
 - + torch.mps
 - + torch.xpu
 - + torch.mtia
 - + torch.mtia.memory
 - + torch.mtia.mtia_graph
 - + Meta device
 - + torch.backends
 - + torch.export
 - + Distributed communication package - torch.distributed
 - + torch.distributed.tensor
 - + Generic Join Context Manager
 - + Torch Distributed Elastic
 - + FullyShardedDataParallel
 - + torch.distributed.fsdp.fully_shard
 - + Tensor Parallelism - torch.distributed.tensor.parallel
 - + Distributed Optimizers
 - + Pipeline Parallelism
 - + PyTorch Symmetric Memory
 - + Distributed Checkpoint - torch.distributed.checkpoint
 - + Probability distributions - torch.distributions
 - + torch.compiler API reference
 - + torch.fft
 - + torch.foreach
 - + torch.func
 - + torch.futures
 - + torch.fx
 - + torch.fx.experimental
 - + torch.hub
 - + torch.linalg
 - + torch.monitor
 - + torch.signal
 - + torch.special
 - + torch.overrides
 - + torch.nativert
 - + torch.package
 - + torch.profiler
 - + torch.nn.init
 - + torch.nn.attention
 - + torch.onnx
 - + torch.optim
 - + Complex Numbers
 - + DDP Communication Hooks
 - + Quantization
 - + Distributed RPC Framework
 - + torch.random

- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCM) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

[]

- * X
- * GitHub
- * PyTorch Forum
- * PyPi

(BUTTON)

- * Install PyTorch
- * User Guide
 - + PyTorch Main Components
 - + torch.compiler
 - + torch.export
 - + Developer Notes
 - + Accelerator Integration
- * Reference API
 - + torch
 - + torch.nn
 - + torch.nn.functional

- + torch.Tensor
- + Tensor Attributes
- + Tensor Views
- + Automatic Mixed Precision package - torch.amp
- + Automatic differentiation package - torch.autograd
- + torch.library
- + torch.accelerator
- + torch.cpu
- + torch.cuda
- + Understanding CUDA Memory Usage
- + torch.mps
- + torch.xpu
- + torch.mtia
- + torch.mtia.memory
- + torch.mtia.mtia_graph
- + Meta device
- + torch.backends
- + torch.export
- + Distributed communication package - torch.distributed
- + torch.distributed.tensor
- + Generic Join Context Manager
- + Torch Distributed Elastic
- + FullyShardedDataParallel
- + torch.distributed.fsdp.fully_shard
- + Tensor Parallelism - torch.distributed.tensor.parallel
- + Distributed Optimizers
- + Pipeline Parallelism
- + PyTorch Symmetric Memory
- + Distributed Checkpoint - torch.distributed.checkpoint
- + Probability distributions - torch.distributions
- + torch.compiler API reference
- + torch.fft
- + torch.foreach
- + torch.func
- + torch.futures
- + torch.fx
- + torch.fx.experimental
- + torch.hub
- + torch.linalg
- + torch.monitor
- + torch.signal
- + torch.special
- + torch.overrides
- + torch.nativert
- + torch.package
- + torch.profiler
- + torch.nn.init
- + torch.nn.attention
- + torch.onnx
- + torch.optim
- + Complex Numbers
- + DDP Communication Hooks
- + Quantization
- + Distributed RPC Framework
- + torch.random
- + torch.masked
- + torch.nested
- + torch.Size
- + torch.sparse
- + torch.Storage
- + torch.testing
- + torch.utils
- + Benchmark Utils - torch.utils.benchmark
- + torch.utils.checkpoint
- + torch.utils.cpp_extension
- + torch.utils.data
- + torch.utils.deterministic
- + JIT Utils - torch.utils.jit
- + torch.utils.dlpack
- + torch.utils.mobile_optimizer
- + torch.utils.model_zoo
- + torch.utils.tensorboard
- + torch.utils.module_tracker
- + Type Info
- + torch.config
- + torch.__future__
- + torch._logging
- + Torch Environment Variables
- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics

- + CPU threading and TorchScript inference
- + CUDA semantics
- + PyTorch Custom Operators Landing Page
- + Distributed Data Parallel
- + Extending PyTorch
- + Extending torch.func with autograd.Function
- + Frequently Asked Questions
- + Getting Started on Intel GPU
- + Gradcheck mechanics
- + HIP (ROCm) semantics
- + Features for large-scale deployments
- + LibTorch Stable ABI
- + LocalTensor Tutorial: Single-Process SPMD Debugging
- + MKLDNN backend
- + Modules
- + MPS backend
- + Multiprocessing best practices
- + Numerical accuracy
- + Out Notes
- + Reproducibility
- + Serialization semantics
- + TensorIterator (Python)
- + Windows FAQ
- * Community
 - + PyTorch Governance | Build + CI
 - + PyTorch Contribution Guide
 - + PyTorch Design Philosophy
 - + PyTorch Governance | Mechanics
 - + PyTorch Governance | Maintainer Guide
 - + PyTorch Governance | Maintainers
 - + viable/strict CI Jobs
- * Tutorials

Go to pytorch.org

Ctrl+K

-
- []
- * X
 - * GitHub
 - * PyTorch Forum
 - * PyPi

Section Navigation

Introduction

- * PyTorch Overview
- * Get Started
- * Learn the Basics

Core Concepts

- * PyTorch Main Components

Torch Compile

- * Torch.compile
 - + Getting Started
 - + Core Concepts
 - o torch.compile Programming Model
 - # Dynamo Core Concepts
 - # Working with Graph Breaks
 - # Non-strict Tracing Programming Model
 - # Dealing with Recompilations
 - # Reducing Compile Time
 - # Reducing Guard Overhead
 - # tparse / TORCH_TRACE
 - # Reporting Issues
 - o Dynamo Overview
 - o PyTorch 2.0 NNModule Support
 - o torch.compile has different autograd semantics
 - + Performance
 - o PyTorch 2.0 Performance Dashboard
 - o TorchInductor GPU Profiling
 - o Profiling to understand torch.compile performance
 - o CUDAGraph Trees
 - + Advanced
 - o Dynamo Deep-Dive
 - o Writing Graph Transformations on ATen IR
 - o Fake tensor
 - o Custom Backends
 - o Dynamic Shapes
 - # Dynamic Shapes Core Concepts
 - # Troubleshooting Dynamic Shapes
 - # Advanced Options to Control Dynamic Behavior

- # Beyond the Basics
 - o Ahead-of-Time Compilation with torch.compile
 - o Complex Number Support in torch.compile
- + Troubleshooting FAQs
 - o tlparsed / TORCH_TRACE
 - o Reporting Issues
 - o torch.compile Troubleshooting
 - o Frequently Asked Questions
- + Reference/API
 - o torch.compiler API reference
 - # torch.compiler.compile
 - # torch.compiler.reset
 - # torch.compiler.nonstrict_trace
 - # torch.compiler.allow_in_graph
 - # torch.compiler.substitute_in_graph
 - # torch.compiler.assume_constant_result
 - # torch.compiler.list_backends
 - # torch.compiler.disable
 - # torch.compiler.set_default_backend
 - # torch.compiler.get_default_backend
 - # torch.compiler.set_stance
 - # torch.compiler.set_enable_guard_collectives
 - # torch.compiler.cudagraph_mark_step_begin
 - # torch.compiler.cudagraph_mark_warmup_incomplete
 - # torch.compiler.is_compiling
 - # torch.compiler.is_dynamo_compiling
 - # torch.compiler.is_exporting
 - # torch.compiler.keep_portable_guards_unsafe
 - # torch.compiler.skip_guard_on_inbuilt_nn_modules_unsafe
 - # torch.compiler.skip_guard_on_all_nn_modules_unsafe
 - # torch.compiler.keep_tensor_guards_unsafe
 - # torch.compiler.skip_guard_on_globals_unsafe
 - # torch.compiler.skip_all_guards_unsafe
 - # torch.compiler.nested_compile_region
 - # torch.compiler.load_cache_artifacts
 - # torch.compiler.load_compiled_function
 - # torch.compiler.save_cache_artifacts
 - # torch.compiler.wrap_numpy
 - o torch.compiler.config
 - o TorchDynamo APIs for fine-grained tracing
 - o TorchInductor and AOTInductor Provenance Tracking
- * Torch.export
 - + torch.export API Reference
 - + torch.export Programming Model
 - + torch.export IR Specification
 - + PT2 Archive Spec
 - + Draft Export
 - + Joint with descriptors
 - + Control Flow Operators
 - o Control Flow - Cond
 - o Control Flow - Switch
 - o Control Flow - While Loop
 - o Control Flow - Scan
 - o Control Flow - Associative Scan
 - o Control Flow - Map
 - + ExportDB
 - o torch.escape-hatch
 - o torch.dynamic-shape
 - o torch.cond
 - o python.closure
 - o torch.dynamic-value
 - o python.data-structure
 - o python.assert
 - o python.control-flow
 - o torch.map
 - o python.builtin
 - o python.object-model
 - o python.context-manager
 - o torch.operator
 - o torch.mutation
 - + AOTInductor: Ahead-Of-Time Compilation for Torch.Export-ed Models
 - o torch._logging
 - # torch._logging.set_logs
 - o AOTInductor Minifier
 - o AOTInductor Debugging Guide
 - + IRs
 - + Dynamic Shapes
 - o Dynamic Shapes Core Concepts
 - o Troubleshooting Dynamic Shapes
 - # Debugging with tlparsed and TORCH_LOGS=dynamic
 - # Troubleshooting GuardOnDataDependentSymNode Errors
 - o Advanced Options to Control Dynamic Behavior

- o Beyond the Basics
 - # The Zero-One Specialization Problem
 - # Backed vs Unbacked Symints
- + Fake tensor
- + Writing Graph Transformations on ATen IR

Developer Notes

- * Developer Notes
 - + Automatic Mixed Precision examples
 - + Autograd mechanics
 - + Broadcasting semantics
 - + CPU threading and TorchScript inference
 - + CUDA semantics
 - + PyTorch Custom Operators Landing Page
 - + Distributed Data Parallel
 - + Extending PyTorch
 - + Extending torch.func with autograd.Function
 - + Frequently Asked Questions
 - + Getting Started on Intel GPU
 - + Gradcheck mechanics
 - + HIP (ROCm) semantics
 - + Features for large-scale deployments
 - + LibTorch Stable ABI
 - + LocalTensor Tutorial: Single-Process SPMD Debugging
 - + MKLDNN backend
 - + Modules
 - + MPS backend
 - + Multiprocessing best practices
 - + Numerical accuracy
 - + Out Notes
 - + Reproducibility
 - + Serialization semantics
 - + TensorIterator (Python)
 - + Windows FAQ

Accelerator Integration

- * Accelerator Integration
 - + Device Management
 - + Accelerator Hooks
 - + Guard
 - + Autoload Mechanism
 - + Operator Registration
 - + Automatic Mixed Precision
 - + Profiler Integration
 - + Distributed Training Integration
 - + CI Integration

- *
- * User Guide
- *
- * Advanced
- * Dynamic Shapes

Rate this Page

? ? ? ? ?

Dynamic Shapes#

Created On: May 19, 2023 | Last Updated On: Aug 18, 2026

This section explains how to work with dynamic shapes in PyTorch, including how to debug and fix common errors, implement support for dynamic shapes in operators, and understand the underlying mechanisms.

Dynamic shapes allow PyTorch models to handle inputs with varying dimensions without recompilation. This enables more flexible models that can process different batch sizes, sequence lengths, or image dimensions in a single compiled artifact. Dynamic shapes work by symbolically tracing tensor dimensions rather than using concrete values, creating a computation graph that adapts to different input shapes at runtime. By default, PyTorch assumes all input shapes to be static.

Typically, deep learning compilers only support static shapes, requiring recompilation for input shape changes.

While this approach covers many use cases, there are situations where this is insufficient:

- * Variable Dimensions - Batch sizes or sequence lengths vary, such as in adaptive batching.
- * Data-Dependent Outputs - Models produce outputs based on input data, like variable bounding boxes in detection models.
- * Sparse Representations - Processing depends on data-varying sparse structures, such as in sparse tensors, jagged tensors, and graph neural networks.

Dynamic shapes do not support dynamic rank programs, programs which input tensors change in dimensionality, as this is uncommon and unnecessarily complex.

What does it mean for a size/integer to be dynamic?#

Dynamic shapes allow avoiding recompilations by making certain dimensions or integers dynamic. For example, if a function `f(x)` is compiled with a static size, it will need recompilation for different sizes:

Note

For simplicity, this example uses `@torch.compile(dynamic=True)`. Note, that this option is not recommended due to it being error prone. For a recommended way of enabling dynamic shapes, see [Enabling Dynamic Behavior](#).

```
import torch
@torch.compile(dynamic=False)
def f(x):
    return x* x.size()[0]

f(torch.rand(10))
f(torch.rand(20))
f(torch.rand(30))
f(torch.rand(40))
```

In the produced output, you can see that four graphs were generated. See the corresponding `tlparse` output

By making the size dynamic, the function can handle various sizes without recompilation:

```
import torch
@torch.compile(dynamic=True)
def f(x):
    return x* x.size()[0]

f(torch.rand(10))
f(torch.rand(20))
f(torch.rand(30))
f(torch.rand(40))
```

With dynamic shapes enabled, only one graph is created. See the corresponding `tlparse` output.

While compilation time differences are minimal for this small example, more complex use cases would show significant performance improvements.

What is a specialization?#

Specialization refers to optimizing a computational graph for specific input shapes by examining shape conditions during control flow. If a branch is taken based on a shape condition, the graph is tailored for that condition. If a new input doesn't meet this condition, the system will recompile the graph.

Specialization allows you to create optimized computational graphs for specific input shapes, which can significantly improve execution speed.

```
import torch
@torch.compile(dynamic=True)
def f(x):
    if x.size()[0] == 10:
        return x * 10

    if x.size()[0] <= 30:
        return x*200

    return x*x.size()[0]

f(torch.rand(10))
f(torch.rand(20))
f(torch.rand(30))
f(torch.rand(40))
f(torch.rand(50))
```

In the code above, we specialize that the graph requires an input size of 10, in which case it will return `x * 10`. If the input size is less than 30, it will return `x * 200`. In the output, you can see that this creates three graphs.

See the corresponding `tlparse` output

This is how graphs created for the above function:
`../../../../images/dynamic_shapes_example_specialization.png`

Enabling Dynamic Behavior#

There are the following ways to make things dynamic:

- * Automatic dynamic
- * User Annotations (preferred)
- * `torch.compile (dynamic=true)` (Not recommended) (for testing only)
- * Advanced Options to Control Dynamic Behavior (for advanced use cases)

Read below about each of these options.

Automatic dynamic#

Automatic dynamic is the default behavior where `torch.compile()` performs the initial compilation assuming static shapes are used, while tracking the input sizes from that first compilation. When a recompile is triggered, it uses this information to identify which dimensions have changed and marks those as dynamic for the second compilation.

User Annotations#

Several APIs allow users to explicitly mark specific inputs by name or code as dynamic. This is useful for avoiding initial compilations that would eventually become dynamic with the previous tools. It is also used to mark elements that do not automatically get marked as dynamic, such as neural network module parameters, and so on. User annotations are the preferred way to enable dynamic shapes.

```
mark_dynamic(tensor, dim, min=min, max=max)#
```

? Warning

`torch._dynamo.mark_dynamic()` must not be called inside any function that is being compiled by `torch.compile()` (for example, a model's `forward()` method or any function it calls).

This function is a tracing-time API. If it is invoked from within compiled code, Dynamo will raise an error

such as:

```
AssertionError: Attempt to trace forbidden callable
```

Correct usage is to call `mark_dynamic` on input tensors before invoking `torch.compile`, for example:

```
torch._dynamo.mark_dynamic(x, 0)
compiled_model = torch.compile(model)
```

The `torch._dynamo.mark_dynamic()` function marks a tensor dimension as dynamic and will fail if it gets specialized. It does not work for integers. Use this function only if you know all graphs in the framework this input converge to a single dynamic graph. Otherwise, you may encounter a misleading constraint violation error. In such cases, consider using `torch._dynamo.maybe_mark_dynamic()`. Currently, `torch._dynamo.mark_dynamic()` does not have precedence over `force_parameter_static_shapes = True` or `force_nn_module_property_static_shapes = True`.

If you know in advance that a particular dimension will be dynamic, you can avoid the initial recompilation by using `torch._dynamo.mark_dynamic(tensor, dim)()`. Additionally, if you already know the minimum and maximum possible values for this dimension, you can specify them with `torch._dynamo.mark_dynamic(tensor, dim, min=min, max=max)()`.

Here is a quick example:

```
import torch

@torch.compile
def f(x):
    return x * x.size()[0]

x = torch.randn(10)
torch._dynamo.mark_dynamic(x, 0)

# first invocation we give it is a tensor marked as dynamic
f(x)
# rest of these invocations will use dynamically compiled code
f(torch.randn(20))
f(torch.randn(30))
f(torch.randn(40))

maybe_mark_dynamic(tensor, dim)#
```

The `torch._dynamo.maybe_mark_dynamic()` function shares all properties with `torch._dynamo.mark_dynamic()` but

does not fail if the size gets specialized. Use it for inputs shared by multiple graphs or if the number of graphs does not converge to one for a specific frame. For instance, in the example above, use `torch._dynamo.maybe_mark_dynamic()` because graphs with sizes 0 and 1 will specialize. However, you can use `torch._dynamo.mark_dynamic()` to ensure you never specialize.

```
mark_unbacked(tensor, dim) #
```

The `torch._dynamo.decorators.mark_unbacked()` function marks a tensor dimension as unbacked. It is unlikely to be the tool you need, but it could be useful if the specialization occurs inside a condition guard_size_oblivious(x), and if using it removes the specialization. Ensure it fixes the specialization and does not introduce a data-dependent error that converts to a graph break at or before the specialization location you are trying to avoid. It might be better to use the next option.

```
Dynamic Allow List (DYNAMIC_SOURCES) #
```

Use the environmental variable `TORCH_COMPILE_DYNAMIC_SOURCES` to pass a configuration list of source names to be marked as dynamic. For example: `TORCH_COMPILE_DYNAMIC_SOURCES=L['x'],L['y']` It's easiest to find these dynamic source names using the PGO artifact in `tlparse`. You can copy and paste the dynamic source names from the PGO artifact. This method works for integers and tensor sizes and has the highest precedence over all other flags that force static shapes. It will not throw an error if what is marked dynamic gets specialized or if the provided input does not exist.

Here is an example:

```
import torch

@torch.compile()
def f(x):
    return x * x.size()[0]

with torch.compiler.config.patch(dynamic_sources="L['x']"):
    f(torch.rand(10))
f(torch.rand(20))
f(torch.rand(30))
f(torch.rand(40))
```

```
Static Source List (STATIC_SOURCES) #
```

The inverse of the dynamic source list: use the environment variable `TORCH_COMPILE_STATIC_SOURCES` (or `torch.compiler.config.static_sources`) to pass a list of source names that should stay static. This is useful when automatic dynamic shapes or PGO makes something dynamic and that dynamism hurts you, for example when it produces a worse kernel. It accepts the same exact-name, regex and :N per-dim syntax as `TORCH_COMPILE_DYNAMIC_SOURCES`, and works for both integers and tensor sizes.

Entries override automatic dynamic shapes, PGO and `dynamic=True`. They do not override an explicit `torch._dynamo.mark_dynamic()`.

Here is an example:

```
import torch

@torch.compile()
def g(x):
    return x * x.size()[0]

with torch.compiler.config.patch(static_sources="L['x']"):
    g(torch.rand(10))
    g(torch.rand(20))
    g(torch.rand(30))

torch.compiler.set_stance ("eager_then_compile") #
```

At times, identifying the appropriate inputs to mark as dynamic can be challenging. If you are willing to accept a performance cost for the first batch, another convenient option is to use the `eager_then_compile` stances, which automatically determine dynamic inputs for you. For more information, see `torch.compiler.set_stance()` and Dynamic Compilation Control with `torch.compiler.set_stance`.

```
torch.compile (dynamic=true) (Not recommended) #
```

This setting forces all sizes and integers to be dynamic, increasing the chance of encountering dynamic shape bugs. Setting this option is not recommended due to it being error prone. It would make every input size

dynamic which may result in performance regressions and ultimately increase compilation time.

PyTorch also provides advanced control options for dynamic shapes, see: [Advanced Options to Control Dynamic Behavior](#).

Where Do I Go From Here?#

If you encounter a framework code bug or an issue with specialization, file an issue so it can be reviewed and potentially improved. If the issue is within your user code, consider whether you are willing to rewrite your code to avoid it. Determine if it affects correctness or if it's a redundant check. If the issue involves a Triton custom kernel with a `constexpr` argument, evaluate whether you can rewrite it to address the problem.

- * [Dynamic Shapes Core Concepts](#)
- * [Troubleshooting Dynamic Shapes](#)
- * [Advanced Options to Control Dynamic Behavior](#)
- * [Beyond the Basics](#)

See also

- * [tlparse documentation](#)
- * [The dynamic shapes manual](#)

Rate this Page

?? ??

(BUTTON) [Send Feedback](#)

[previous](#)

[Custom Backends](#)

[next](#)

[Dynamic Shapes Core Concepts](#)

Built with the [PyData Sphinx Theme 0.15.4](#).

[previous](#)

[Custom Backends](#)

[next](#)

[Dynamic Shapes Core Concepts](#)

On this page

- * [What does it mean for a size/integer to be dynamic?](#)
- * [What is a specialization?](#)
- * [Enabling Dynamic Behavior](#)
 - + [Automatic dynamic](#)
 - + [User Annotations](#)
 - o `mark_dynamic(tensor, dim, min=min, max=max)`
 - o `maybe_mark_dynamic(tensor, dim)`
 - o `mark_unbacked(tensor, dim)`
 - o [Dynamic Allow List \(DYNAMIC_SOURCES\)](#)
 - o [Static Source List \(STATIC_SOURCES\)](#)
 - o `torch.compiler.set_stance ("eager_then_compile")`
 - + `torch.compile (dynamic=true)` (Not recommended)
- * [Where Do I Go From Here?](#)

[Edit on GitHub](#)

[Show Source](#)

[PyTorch Libraries](#)

- * [ExecuTorch](#)
- * [Helion](#)
- * [torchao](#)
- * [kineto](#)
- * [torch.titan](#)
- * [TorchRL](#)
- * [torchvision](#)
- * [torchaudio](#)
- * [tensordict](#)
- * [PyTorch on XLA Devices](#)

[Docs](#)

[Access comprehensive developer documentation for PyTorch](#)
[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers
[View Tutorials](#)

Resources

Find development resources and get your questions answered
[View Resources](#)

Stay in touch for updates, event info, and the latest news

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).

*
*
*
*
*

(C) PyTorch. Copyright (C) The Linux Foundation(R). All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Trademark Usage](#). [Privacy Policy](#).

To analyze traffic and optimize your experience, we serve cookies on this site. By clicking or navigating, you agree to allow our usage of cookies. As the current maintainers of this site, Facebook's Cookies Policy applies. [Learn more](#), including about available controls: [Cookies Policy](#).
(BUTTON)

(C) Copyright PyTorch Contributors.

Created using Sphinx 7.2.6.

Built with the PyData Sphinx Theme 0.15.4.